

21. Low-Level vs. High-Level: What are the key differences? Provide one example of each.

- Low-Level Languages: These are machine-oriented and closer to binary (0s and 1s). They provide direct control over hardware but are difficult for humans to read. Example: Assembly Language.
- High-Level Languages: These use English-like syntax, making them user-friendly and portable across different platforms. They require a compiler or interpreter to run. Example: Java or Python.

22. Compiler vs. Interpreter: Explain the difference. Which one does Java use?

- Compiler: Translates the entire source code into machine code at once before execution.
- Interpreter: Translates and executes code line-by-line.
- Java Model: Java uses a hybrid approach. It first uses a compiler to turn source code into Bytecode, and then the Java Virtual Machine (JVM) interprets (or JIT compiles) that Bytecode into machine-specific instructions. This achieves platform independence.

23. Bytecode: What role does it play in "Write Once, Run Anywhere"?

Bytecode is a highly optimized set of instructions that is not specific to any processor. Because Java programs are compiled into Bytecode (.class files) rather than machine code, any device with a JVM installed can run the same file, regardless of the underlying hardware or operating system.

24. JVM: What is the Java Virtual Machine (JVM)?

The JVM is an abstract computing machine that provides a runtime environment for Java Bytecode to execute. It acts as a bridge between the compiled Java code and the actual hardware.

25. When to use if-else if-else vs. switch?

- if-else if: Best for complex conditions, ranges (e.g., score > 90), or checking multiple different variables.
- switch: Best for testing a single variable against a list of specific constant values (like menu options or days of the week). It is often more readable and faster for many fixed cases.

26. Loop Types: Describe the primary use cases.

- for loop: Used when the number of iterations is known in advance (e.g., repeating 10 times).
- while loop: Used when the number of iterations is unknown and depends on a condition being true.

- do-while loop: Similar to while, but guarantees the code block runs at least once because the condition is checked after the execution.

27. break vs. continue: What is the difference?

- break: Immediately exits the loop entirely. (Example: Stopping a search once an item is found).
- continue: Skips the current iteration and jumps to the next cycle of the loop. (Example: Skipping even numbers in a list).

28. Even or Odd

```
int num = 7; // Example input
if (num % 2 == 0) {
    System.out.println("Even");
} else {
    System.out.println("Odd");
}
```

29. Largest of Three Numbers

```
int a = 10, b = 25, c = 15;
if (a >= b && a >= c) System.out.println(a + " is largest");
else if (b >= a && b >= c) System.out.println(b + " is largest");
else System.out.println(c + " is largest");
```

30. Leap Year Checker

```
int year = 2024;
if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
    System.out.println("Leap Year");
} else {
    System.out.println("Not a Leap Year");
}
```

31. Grade Calculator

```
int score = 85;
if (score >= 90) System.out.println("A");
else if (score >= 80) System.out.println("B");
else if (score >= 70) System.out.println("C");
else if (score >= 60) System.out.println("D");
else System.out.println("F");
```

32. Day of the Week (switch)

```
int day = 3;
switch(day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    case 3: System.out.println("Wednesday"); break;
    // ... add cases up to 7 ...
    default: System.out.println("Invalid day");
}
```

33. Sum of First N Natural Numbers

```
int n = 10, sum = 0;

for (int i = 1; i <= n; i++) {
    sum += i;
}
System.out.println("Sum: " + sum);
```

34. Factorial Calculation

```
int n = 5, fact = 1;
while (n > 0) {
    fact *= n;
    n--;
}
System.out.println("Factorial: " + fact);
```

35. Number Guessing Game (Conceptual Snippet)

```
int target = (int)(Math.random() * 100) + 1;
int guess;
do {
    System.out.print("Guess: ");
    guess = sc.nextInt();
    if (guess > target) System.out.println("Too high!");
    else if (guess < target) System.out.println("Too low!");
} while (guess != target);
System.out.println("Correct!");
```

36. Multiplication Table

```
int n = 5;
for (int i = 1; i <= 10; i++) {
```

```
    System.out.println(n + " x " + i + " = " + (n * i));  
}
```

37. Count Digit

```
int num = 12345, count = 0;  
while (num != 0) {  
    num /= 10;  
    count++;  
}  
System.out.println("Digits: " + count);
```

38. Prime Number Checker

```
int num = 29;  
boolean isPrime = true;  
for (int i = 2; i <= num / 2; i++) {  
    if (num % i == 0) {  
        isPrime = false;  
        break;  
    }  
}  
System.out.println(isPrime ? "Prime" : "Not Prime");
```

39. Print Even Numbers (1-50 using continue)

```
for (int i = 1; i <= 50; i++) {  
    if (i % 2 != 0) continue;  
    System.out.print(i + " ");  
}
```

40. Reverse a String

```
String word = "Java";  
String reversed = "";  
for (int i = word.length() - 1; i >= 0; i--) {  
    reversed += word.charAt(i);  
}  
System.out.println(reversed);
```